

Platform and Cost Architecture

**Common Patterns.
Clear Fixes.**

50 engineering patterns covering workspace design, cost management, CI/CD with Asset Bundles, secrets management, environment strategy, monitoring and observability, and access control in Databricks. Patterns every platform engineer must recognise.

INTRODUCTION

The platform decisions nobody notices until they go wrong.

Platform and cost architecture decisions are made once and rarely revisited. A workspace structure that works for five engineers becomes unmanageable for fifty. A service principal that was given broad access for convenience becomes a security incident waiting to happen. A cost monitoring gap that seemed acceptable in month one produces a surprise bill in month six. These patterns are not code problems. They are architecture problems that accumulate silently.

This book covers seven chapters. Chapter 1 covers workspace topology decisions that become significant operational problems as teams and workloads grow on Azure. Chapter 2 covers Databricks cost management and the gaps in visibility through the Azure consumption bill that allow spending to grow unchecked. Chapter 3 covers Asset Bundle deployments and the failure modes of CI/CD pipelines that promote code between workspaces. Chapter 4 covers secrets management including Azure Key Vault rotation. Chapter 5 covers the gaps between development and production environments. Chapter 6 covers monitoring and observability and the specific ways pipelines fail silently while appearing healthy. Chapter 7 covers service principal and user credential patterns including Microsoft Entra ID federation and the network controls that govern who can reach the workspace.

Every scenario follows the same structure: the situation, the wrong thinking, what is actually happening, and the fix. Platform problems are particularly expensive because they affect every workload built on top of the platform rather than a single pipeline.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 1

Workspace Design

Seven scenarios covering the workspace topology decisions that look reasonable at small scale and become significant operational problems as teams and workloads grow on Azure.

01

WORKSPACE DESIGN

You created one workspace for everything. Teams are overwriting each other's work.

THE SITUATION

A single Databricks workspace serves all teams. Engineers from different teams work in the same workspace on notebooks with similar names. A data engineer accidentally overwrites a production notebook while saving changes to a development version with the same name. Another team's scheduled jobs start failing because a cluster configuration was modified by an engineer from a different team.

WRONG THINKING

Engineers create one workspace to simplify administration. A single workspace reduces management overhead and allows all teams to share resources. The coordination problems that emerge when unrelated teams share a workspace are not anticipated.

WHAT IS ACTUALLY HAPPENING

A single workspace with no structural separation between teams creates a shared namespace where accidental overwrites, resource conflicts, and unintended modifications are possible. Notebooks, clusters, and jobs in the same workspace are visible to all workspace users depending on permission settings. Without folder-based isolation, teams working on similar problems create name collisions. Without cluster policies, any team can modify shared cluster configurations.

THE FIX

Separate teams into distinct workspace folders or separate workspaces based on the level of isolation required. For organisations where teams have independent data domains and different governance requirements, separate workspaces with dedicated service principals provide the cleanest isolation. For smaller organisations, use folder-based organisation within a shared workspace combined with strict naming conventions and cluster policies to reduce cross-team interference.

02

WORKSPACE DESIGN

You created a workspace per team. Sharing data and clusters became a coordination nightmare.

THE SITUATION

Each team has its own Databricks workspace. Teams that need to share data or collaborate on joint projects find it difficult. Sharing a Delta table from workspace A to workspace B requires complex Delta Sharing or data export configurations. Clusters cannot be shared across workspaces. Every cross-team integration requires platform engineering involvement.

WRONG THINKING

Engineers create per-team workspaces for isolation and autonomy. Each team owning their workspace provides governance and independence. The coordination overhead of cross-workspace data sharing is not considered during the initial design.

WHAT IS ACTUALLY HAPPENING

Workspace-level isolation prevents direct data sharing through standard SQL or Delta table references. Cross-workspace access requires Delta Sharing, external table configurations with shared storage credentials, or data export pipelines. For organisations where cross-team data sharing is frequent, per-workspace isolation creates more coordination overhead than it eliminates through isolation.

THE FIX

Design workspace topology based on governance boundaries rather than team structure. For most organisations, a small number of workspaces by environment such as dev, staging, and production provides the right balance of isolation and collaboration. Use Unity Catalog catalogs and schemas within shared workspaces to provide team-level data isolation without workspace-level network separation. Reserve per-team workspaces for cases where genuine regulatory or security isolation is required.

03

WORKSPACE DESIGN

Your dev and prod are in the same workspace. A developer deleted a production table.

THE SITUATION

Development and production workloads share a single Databricks workspace. A developer cleaning up test tables accidentally deletes a production Gold table. The deletion is permanent because no soft-delete mechanism was in place. Recovery requires reprocessing 3 days of pipeline history.

WRONG THINKING

Engineers use one workspace for all environments to reduce management overhead. Developers are trusted to distinguish between dev and prod objects based on naming conventions. The risk of accidental deletion without a technical enforcement boundary is accepted implicitly.

WHAT IS ACTUALLY HAPPENING

A single workspace provides no technical enforcement of the dev/prod boundary. Naming conventions are enforced by team discipline rather than access controls. Any user with DROP TABLE permission in the workspace can delete any table regardless of whether it is a development or production object. Accidental deletions, overwrites, and configuration changes in production are an expected outcome of shared workspace environments.

THE FIX

Separate development and production into distinct workspaces with separate service principals, access controls, and Delta table storage paths. This is the minimum viable environment separation for platforms serving production business workloads. Use Databricks deployment pipelines or Asset Bundles to promote code from dev to prod in a controlled way rather than having developers work directly in the production workspace.

04

WORKSPACE DESIGN

You migrated to a new workspace. Service principal permissions did not migrate with it.

THE SITUATION

A Databricks workspace is migrated to a new Azure region. All jobs, notebooks, and clusters are recreated in the new workspace. After migration, all jobs begin failing with authentication errors. Investigation reveals that service principal permissions granted in the old workspace were not migrated.

WRONG THINKING

Engineers migrate workspace assets including jobs, notebooks, and cluster configurations. Service principal permissions are workspace-level configurations that are assumed to migrate as part of the workspace export and import process.

WHAT IS ACTUALLY HAPPENING

Service principal permissions in Databricks are workspace-level assignments. Exporting workspace assets does not export the workspace-level permission assignments for service principals. Each service principal must have its permissions explicitly re-granted in the new workspace after migration. Additionally on Azure several adjacent configurations are independent of workspace assets and must be reconfigured separately: the Azure Databricks access connector and its managed identity role assignments on ADLS Gen2 storage accounts, Unity Catalog metastore attachments, IP access lists and Private Link configurations, and the workspace's Microsoft Entra ID federation settings. None of these travel with a workspace export.

THE FIX

Before migrating a workspace, document all service principal permission assignments including workspace roles, cluster permissions, job permissions, and Unity Catalog grants. On Azure, also document the access connector resource ID, the managed identity role assignments on each storage account, the IP access list configuration, the Private Link private endpoint configuration, and the Unity Catalog metastore region. Include all of these as mandatory steps in the migration runbook. After migration, test each service principal by running a representative job before redirecting production traffic to the new workspace. Treat identity, network, and storage credential migration as separate workstreams from asset migration.

05

WORKSPACE DESIGN

Your workspace has 500 users. Notebook search returns results from other teams.

THE SITUATION

A shared Databricks workspace has grown to 500 users across many teams. Engineers searching for notebooks find results from other teams mixed in with their own. New engineers cannot distinguish which notebooks belong to their team and which belong to others. The workspace has become a disorganised shared filesystem.

WRONG THINKING

Engineers add users to the workspace as teams grow without establishing a folder structure or naming convention. The workspace is treated as a flat namespace where all notebooks coexist. Organisation is expected to emerge naturally as the workspace grows.

WHAT IS ACTUALLY HAPPENING

Databricks workspace notebooks are stored in a hierarchical folder structure. Without an enforced folder convention, notebooks accumulate in personal and shared folders with no consistent organisation. At 500 users, the volume of notebooks makes search results ambiguous and navigation by folder structure impossible. The lack of structure increases the time required for any task involving finding or sharing existing work.

THE FIX

Establish a workspace folder structure based on teams and data domains before the workspace grows beyond 20 engineers. Create top-level folders per team or domain and assign folder ownership to each team. Set folder-level permissions so teams manage their own folder contents. Implement a naming convention for notebooks that includes the team prefix, environment, and purpose. Retroactively organising a workspace with 500 users requires significant effort that could have been avoided with upfront structure.

06

WORKSPACE DESIGN

You separated workspaces by environment. Promoting code between them requires manual steps.

THE SITUATION

Separate dev and prod workspaces are maintained for environment isolation. Promoting a tested change from dev to prod requires manually recreating or updating jobs, clusters, and notebooks in the production workspace. Each promotion involves multiple manual steps and human error is common.

WRONG THINKING

Engineers separate environments by workspace for isolation and assume code promotion between workspaces will be manageable. Manual promotion is accepted as the cost of environment isolation.

WHAT IS ACTUALLY HAPPENING

Manual code promotion between workspaces is inherently error-prone and does not scale as the number of pipelines and engineers grows. A manual process that works for 5 jobs becomes unmanageable for 50. Each manual step is an opportunity for inconsistency between environments. Over time, dev and prod diverge in configuration details that manual promotion misses.

THE FIX

Implement automated deployment using Databricks Asset Bundles to define workspace resources as code and deploy them consistently across environments. Asset Bundles allow the same pipeline definition to be deployed to dev and prod with environment-specific parameter overrides. Integrate Asset Bundle deployments into a CI/CD pipeline that automatically promotes tested changes from dev to prod through a defined approval process. Manual promotion should be eliminated from production deployment workflows.

07

WORKSPACE DESIGN

Your workspace admin left the company. There is no secondary admin and nobody can manage access.

THE SITUATION

The only workspace admin for a Databricks workspace leaves the company. Their account is deactivated. The remaining team members cannot add new users, create clusters, modify permissions, or perform any admin tasks. The platform is effectively frozen until a new admin is granted access through an escalation process that takes several days.

WRONG THINKING

Engineers designate a single workspace admin for operational simplicity. One person managing access reduces the risk of conflicting admin actions. The business continuity risk of having a single point of admin access is not considered.

WHAT IS ACTUALLY HAPPENING

A workspace with a single admin is a single point of failure for platform administration. When that person is unavailable whether through departure, illness, or accident, all administrative tasks requiring workspace admin access are blocked. The organisation has no ability to manage its own platform until the admin access is restored through a lengthy escalation process.

THE FIX

Always maintain at least two workspace admins from different teams or with different reporting lines. Use a shared service account as a permanent workspace admin in addition to named individual admins. Document the process for recovering admin access through Databricks account admin as a contingency plan. Review workspace admin assignments quarterly and update when individuals change roles or leave the organisation.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 2

Cost Management

Eight scenarios covering the specific ways Databricks costs accumulate without visibility through the Azure consumption bill and the monitoring gaps that allow spending to grow unchecked.

08

COST MANAGEMENT

Your Databricks bill doubled last month. You have no visibility into which jobs caused it.

THE SITUATION

The monthly Databricks bill is twice the previous month's amount. The data engineering team has no per-job or per-team cost attribution and cannot identify which workloads caused the increase. The investigation requires manually reviewing cluster event logs and job run histories which takes days.

WRONG THINKING

Engineers focus on building pipelines and trust that costs will be manageable. Cost monitoring is treated as a finance team concern rather than an engineering concern. Cost attribution infrastructure is not built during the initial platform setup.

WHAT IS ACTUALLY HAPPENING

Without cost attribution tagging and monitoring, Databricks DBU consumption is reported as a single total per workspace or account. When costs increase unexpectedly, identifying the responsible workload requires reconstructing cost from job run histories and cluster utilisation logs. This retroactive analysis is slow and often incomplete. The investigation takes longer than it would have taken to set up cost monitoring proactively.

THE FIX

Implement cost attribution from day one by tagging all clusters and SQL warehouses with team, project, and environment tags. Configure the Databricks `system.billing.usage` system table or the Azure Cost Management views filtered by Databricks resource tags to provide per-tag cost breakdown. On Azure both the DBU charge and the underlying Azure VM consumption appear on the same Azure subscription bill, so cross-reference the two views to get a complete picture. Set up weekly cost reports that attribute spending to each team and project. Establish cost budgets per team and configure Azure Cost Management budget alerts when spending exceeds defined thresholds. Treat cost monitoring as a production requirement alongside performance and reliability monitoring.

09

COST MANAGEMENT

You optimised all your jobs. The bill is the same. Idle clusters are the actual cost.

THE SITUATION

A cost optimisation initiative focuses on improving job efficiency. All major jobs are optimised to run faster and use fewer DBUs. After the optimisation the monthly bill is unchanged. Investigation reveals that all-purpose clusters running continuously with no active jobs are consuming more DBUs than all the optimised jobs combined.

WRONG THINKING

Engineers identify running jobs as the primary cost driver and optimise them. The cost of idle infrastructure is not considered because idle resources seem like they should cost nothing or very little.

WHAT IS ACTUALLY HAPPENING

All-purpose clusters in Databricks charge DBUs for every hour they run regardless of whether jobs are executing. A cluster running 24 hours a day with jobs only running for 4 of those hours pays for 24 hours of DBUs. For all-purpose cluster DBU rates which are 2 to 3 times higher than job compute rates, idle cluster time can dominate total cost. Optimising job execution time reduces the cost of active processing but has no effect on idle cluster charges.

THE FIX

Audit all-purpose cluster utilisation to identify clusters that are running with low utilisation. Set auto-termination on all all-purpose clusters to terminate after a defined idle period. Review clusters that have auto-termination set but are being kept alive by periodic background activity such as notebook health checks or scheduled light queries. Switch production batch workloads from all-purpose clusters to job clusters which only run during job execution. The combined impact of auto-termination and job cluster migration typically produces larger cost savings than job execution optimisation alone.

10

COST MANAGEMENT

Your all-purpose clusters are running 24 hours a day. Auto-termination is set but not triggering.

THE SITUATION

All-purpose clusters have auto-termination configured for 30 minutes of inactivity. Despite this setting, several clusters run continuously around the clock. The clusters show as active in the workspace UI but no jobs are running on them.

WRONG THINKING

Engineers configure auto-termination expecting it to terminate idle clusters automatically. The configuration is verified in the cluster settings. Clusters that remain running despite the configuration are assumed to be actively used by someone.

WHAT IS ACTUALLY HAPPENING

Auto-termination triggers based on the absence of Spark commands. Certain workspace features including notebook auto-complete queries, variable explorer refreshes, and scheduled notebook health checks submit lightweight Spark commands that count as activity. A cluster with attached notebooks that are open in users' browsers may never reach the inactivity threshold because background notebook activity resets the timer continuously. The cluster appears idle to an observer but is technically active from Databricks' perspective.

THE FIX

Add a maximum lifetime configuration to all-purpose clusters in addition to auto-termination. Maximum lifetime terminates the cluster after an absolute duration regardless of activity, preventing background activity from keeping clusters alive indefinitely. Set maximum lifetime to 12 to 16 hours for development clusters that should not run overnight. Monitor clusters that consistently run at near-zero utilisation despite auto-termination settings to identify the background activity keeping them alive.

11

COST MANAGEMENT

You moved to serverless compute to reduce cost. Cost increased for short interactive jobs.

THE SITUATION

A team migrates interactive development workloads from all-purpose clusters to serverless compute expecting cost savings. After migration, the monthly cost for interactive development increases. Serverless compute is expected to be cheaper due to per-second billing.

WRONG THINKING

Engineers choose serverless for interactive work based on the per-second billing model which seems more efficient than paying for idle cluster time. The DBU rate difference between serverless and all-purpose is not calculated before migration.

WHAT IS ACTUALLY HAPPENING

Serverless compute has a higher per-DBU rate than all-purpose cluster compute. For short interactive queries that take seconds to execute, per-second billing reduces the idle time cost but the higher DBU rate for each second of active execution may cost more than the equivalent time on a lower-rate all-purpose cluster. For workloads with very short execution times and long idle periods between them, serverless is cost-effective. For workloads with frequent interactive queries throughout the day, the higher rate per active second can exceed the all-purpose cluster cost.

THE FIX

Calculate the expected cost of serverless versus all-purpose for the specific usage pattern before migrating. For interactive development with frequent short queries throughout the day, a right-sized all-purpose cluster with auto-termination may be cheaper than serverless despite paying for some idle time. Use the Databricks pricing calculator with actual observed query frequency and duration data before making the migration decision.

12

COST MANAGEMENT

Your photon-enabled jobs cost more than non-Photon jobs doing the same work.

THE SITUATION

A team enables Photon on job clusters expecting cost savings from faster query execution. After one month, Photon-enabled jobs cost more than the equivalent non-Photon jobs despite running faster. The jobs complete in less time but the total DBU cost is higher.

WRONG THINKING

Engineers assume faster execution with Photon automatically reduces cost because less time is spent running the job. Faster jobs should use fewer DBUs.

WHAT IS ACTUALLY HAPPENING

Photon-enabled clusters charge a higher DBU rate than standard clusters. If a Photon job runs 30 percent faster but charges 50 percent more per DBU, the total cost is higher despite the shorter execution time. The cost benefit of Photon depends on the ratio of performance improvement to rate increase for the specific workload. For workloads where Photon provides modest performance gains, the higher rate may exceed the savings from reduced duration.

THE FIX

Calculate the total DBU cost of Photon versus non-Photon for specific workloads by multiplying execution time by DBU rate for each configuration. Photon delivers cost savings when the performance improvement percentage exceeds the rate premium percentage. Profile Photon performance gains for each major workload before enabling it across all job clusters. Enable Photon selectively on workloads where the measured performance gain justifies the rate premium.

13

COST MANAGEMENT

You tagged all clusters for cost attribution. The tags are not appearing in the billing export.

THE SITUATION

All clusters are configured with team and project tags for cost attribution. The monthly billing export is analysed and the tags are absent from the cost records. Cost attribution is still not possible despite the tagging effort.

WRONG THINKING

Engineers add tags to cluster configurations in the Databricks UI expecting the tags to propagate to the cloud provider billing export. Tags in the Databricks cluster configuration should appear in the underlying VM billing data.

WHAT IS ACTUALLY HAPPENING

Databricks cluster tags propagate to the Azure VM tags on the underlying compute, which appear in Azure Cost Management cost analysis. However the propagation is not instantaneous and there can be a delay between when a tag is set in Databricks and when it appears on newly provisioned VMs. For clusters that were already running when tags were added, the tags do not apply until the cluster is restarted. Additionally Azure has tag key and value format restrictions (Azure tag keys cannot start with `microsoft`, `azure`, or `windows`, and have length and character limits) that may cause some Databricks tags to be silently rejected.

THE FIX

Restart all clusters after adding tags to ensure the tags are applied to the underlying Azure VMs. Verify tag propagation by checking the Azure portal for the actual VM scale set instances backing the Databricks cluster and confirming the tags appear there. Test a small set of tags before rolling out comprehensive tagging to identify any Azure tag format issues. Use the Databricks `system.billing.usage` system table rather than relying solely on Azure Cost Management for Databricks-specific cost attribution; the system table provides tag-level breakdown without depending on cloud tag propagation timing.

14

COST MANAGEMENT

One team is consuming 70 percent of your Databricks spend. You discovered this at month end.

THE SITUATION

A monthly cost review reveals that one team is responsible for 70 percent of total Databricks spending. The team was running large clusters for exploratory work without realising the cost implications. By the time the cost is discovered, the team has already consumed a significant portion of the platform budget for the month.

WRONG THINKING

Platform engineers assume teams understand the cost implications of their compute choices. Cost visibility is provided through monthly reports which are too infrequent to catch in-period overspending.

WHAT IS ACTUALLY HAPPENING

Monthly cost reviews are too infrequent to detect overspending before it accumulates. A team that starts a cost-intensive workload on the 1st of the month will consume the budget for 4 weeks before the monthly report reveals the problem. Without real-time or weekly cost visibility per team, cost anomalies are discovered after the damage is done.

THE FIX

Implement weekly cost reports per team distributed to team leads. Configure real-time cost alerts that trigger when a team's weekly spending exceeds a defined threshold. Use cluster policies to restrict maximum cluster size per team, preventing any single team from provisioning compute that would cause significant budget impact without a change review process. Treat cost anomaly detection as an operational monitoring requirement with the same urgency as performance or reliability alerts.

15

COST MANAGEMENT

Your cost alert fired. By the time you acted, you had already exceeded the budget by 40 percent.

THE SITUATION

A cost alert is configured to fire when monthly spending reaches the budget threshold. The alert fires and the team investigates. By the time the investigation is complete and workloads are stopped, the monthly spend has exceeded the budget by 40 percent because several large jobs continued running during the investigation period.

WRONG THINKING

Engineers configure a budget threshold alert expecting it to provide enough warning to take action before the budget is exceeded. An alert at the budget threshold is assumed to provide sufficient lead time.

WHAT IS ACTUALLY HAPPENING

An alert that fires when the budget is reached does not prevent the budget from being exceeded. If high-cost workloads are running when the alert fires, those workloads continue accumulating cost during the investigation and response period. For workloads consuming budget rapidly, the gap between alert firing and workload termination can represent a significant percentage of total overage.

THE FIX

Set cost alerts at 70 and 90 percent of the budget threshold rather than at 100 percent. The 70 percent alert provides early warning for investigation without urgency. The 90 percent alert triggers immediate workload review. Implement automatic workload throttling or cluster termination when the 100 percent threshold is crossed to prevent runaway overage. Design the budget and alert system to stop the bleeding automatically rather than relying on manual intervention after the budget is already exhausted.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 3

CI/CD with Asset Bundles

Seven scenarios covering Asset Bundle deployment failures, environment configuration gaps, and the specific ways automated deployments create inconsistencies that are difficult to detect.

16

ASSET BUNDLES

You deployed a Databricks Asset Bundle. The job is pointing to the wrong cluster config.

THE SITUATION

An Asset Bundle is deployed to production. The deployed job shows a Running status on the first run but is using a different cluster configuration than intended. The cluster uses the development configuration rather than the production configuration specified in the bundle.

WRONG THINKING

Engineers define environment-specific cluster configurations in the Asset Bundle and assume the deployment picks up the correct configuration for the target environment automatically.

WHAT IS ACTUALLY HAPPENING

Asset Bundle environment targets map to specific configuration overrides defined in the bundle's `databricks.yml`. If the production target is not correctly specified or if the variable substitution for cluster configuration references the wrong variable, the deployed job uses a fallback or default cluster configuration rather than the intended production one. Variable substitution errors in Asset Bundles can silently produce deployments that appear successful but use incorrect configurations.

THE FIX

Validate deployed job configurations after every Asset Bundle deployment by reading back the deployed job definition using the Databricks Jobs API and comparing it against the expected configuration. Add a post-deployment validation step to the CI/CD pipeline that checks cluster type, size, and access mode against the expected values. Never assume the deployment used the correct configuration without verification.

17

ASSET BUNDLES

Your bundle deployment succeeded. The job in the workspace was not updated.

THE SITUATION

An Asset Bundle deployment completes successfully with no errors. The job in the target workspace appears to be the same version as before the deployment. The new code changes are not reflected in the deployed job.

WRONG THINKING

Engineers run the bundle deployment command and check the exit code. A zero exit code indicates success and the job is assumed to have been updated with the new configuration.

WHAT IS ACTUALLY HAPPENING

Asset Bundle deployments use an idempotent deployment model. If the bundle determines that the existing workspace configuration matches the bundle definition, it may skip the update and report success without making any changes. This can happen if the bundle hash matches the previously deployed state or if a deployment error was silently swallowed. The deployment reports success but no update was applied.

THE FIX

Add a version identifier or timestamp to the bundle configuration that changes with every deployment, forcing an actual update even when the logical configuration has not changed. After deployment, verify the deployed job's last modified timestamp has updated and check the job definition directly using the Databricks API. Include a deployment verification step in the CI/CD pipeline that confirms the deployed version matches the expected version.

18

ASSET BUNDLES

You used bundle variables for environment config. Production is using dev variable values.

THE SITUATION

An Asset Bundle uses variables for environment-specific configuration such as cluster sizes, table names, and storage paths. The bundle is deployed to production but production jobs are reading from development tables and writing to development storage paths.

WRONG THINKING

Engineers define variables in the bundle and set environment-specific values in the target configuration. The production deployment is assumed to use production variable values automatically.

WHAT IS ACTUALLY HAPPENING

Bundle variable resolution depends on the target configuration being correctly specified and the deployment command using the correct target. If the deployment command does not explicitly specify the production target, it may use the default target which could be development. Additionally if production variable values are not defined for a variable, the bundle falls back to the default value which may be the development value.

THE FIX

Always specify the explicit target in bundle deployment commands using the `--target` flag. Define all variables with explicit values for every target environment with no reliance on default values for production deployments. Add a pre-deployment check that prints all resolved variable values for the target environment and requires explicit confirmation before deploying to production. Never use implicit targets or rely on default variable values in production deployments.

19

ASSET BUNDLES

Your bundle includes a notebook. Changes to the notebook are not reflected after deployment.

THE SITUATION

An Asset Bundle references a notebook as a task source. The notebook code is updated and the bundle is redeployed. The job continues executing the old version of the notebook. The notebook changes are not picked up by the deployment.

WRONG THINKING

Engineers update the notebook and redeploy the bundle expecting the updated notebook to be used by the deployed job.

WHAT IS ACTUALLY HAPPENING

Asset Bundles can reference notebooks by local path or by workspace path. When a notebook is referenced by workspace path and the notebook already exists in the workspace, the bundle deployment may not overwrite it if the deployment strategy is set to bind rather than deploy. Alternatively if the bundle deployment uploads the notebook but the job task references the workspace path of the old notebook rather than the newly deployed one, the job uses the old version.

THE FIX

Configure notebook tasks in Asset Bundles to reference notebooks by the bundle-managed path rather than a pre-existing workspace path. Use the bundle source reference mode which deploys the notebook as part of the bundle and ensures the job always uses the bundle-deployed version. Verify the notebook task source path in the deployed job definition matches the expected bundle-deployed path after each deployment.

20

ASSET BUNDLES

Bundle validation passes locally. Deployment fails in CI with a different error.

THE SITUATION

An Asset Bundle validates successfully when run locally using `databricks bundle validate`. When the same bundle is deployed through a CI pipeline, the deployment fails with an error that does not occur locally.

WRONG THINKING

Engineers validate the bundle locally before pushing to CI and assume a locally-passing validation guarantees CI deployment success. The validation checks are assumed to be equivalent in both environments.

WHAT IS ACTUALLY HAPPENING

Local bundle validation checks the bundle syntax and configuration structure but does not validate against the actual workspace configuration. CI deployment runs against the workspace and may encounter resource limits, permission restrictions, cluster type availability, or workspace-level configuration differences that local validation cannot detect. Additionally CI environments often use different service principal credentials than local development, which may have different workspace permissions.

THE FIX

Run bundle deployment in a staging workspace as part of the CI validation step before deploying to production. This ensures deployment-time errors are caught in CI rather than production. Add workspace-specific validation checks to the CI pipeline that verify the service principal has the required permissions for all bundle operations. Document environment-specific requirements for bundle deployment and verify them in the CI pipeline rather than relying solely on local validation.

21

ASSET BUNDLES

You destroyed a bundle. It deleted resources that were not created by the bundle.

THE SITUATION

A bundle destroy command is run to clean up a decommissioned pipeline. After the destroy, several jobs and clusters that were not created by the bundle are missing from the workspace. The destroy operation deleted workspace resources that the bundle had adopted during previous deployments.

WRONG THINKING

Engineers run bundle destroy to cleanly remove bundle-managed resources. Resources not created by the bundle are assumed to be protected from the destroy operation.

WHAT IS ACTUALLY HAPPENING

Asset Bundles can adopt existing workspace resources through the deployment process. When a bundle is deployed to a workspace that already has a job with the same name, the bundle may adopt that job as a bundle-managed resource. Subsequent destroy operations treat all adopted resources as bundle-managed and delete them. Resources adopted into the bundle are deleted by destroy even if they existed before the bundle was created.

THE FIX

Review the list of bundle-managed resources using `databricks bundle summary` before running destroy. Confirm the list includes only resources that should be deleted. If the list includes resources that should be preserved, remove them from the bundle configuration without deleting them from the workspace before running destroy. Document which resources are bundle-managed and which are independently managed to prevent accidental adoption of critical resources.

22

ASSET BUNDLES

Your bundle deploys correctly to dev. The production deployment silently uses wrong permissions.

THE SITUATION

An Asset Bundle deploys successfully to the development workspace. The same bundle deployed to production runs without errors but job executions fail with permission denied errors. The permission denied errors indicate the production job is using a different service principal than intended.

WRONG THINKING

Engineers validate the bundle in dev and assume production deployment uses equivalent configuration. A successful dev deployment with the correct service principal is assumed to confirm the production deployment will also use the correct identity.

WHAT IS ACTUALLY HAPPENING

Asset Bundles reference service principals through configuration that may resolve differently in dev and prod if the service principal identifiers or names are environment-specific. If the production target configuration does not explicitly specify the service principal and falls back to the dev configuration, the production job runs with the dev service principal which may not have the required production permissions.

THE FIX

Define service principal identity explicitly in every environment target configuration rather than relying on default or inherited values. Use environment-specific variable references for service principal identifiers. After deploying to production, verify the job run identity by checking the job's `run_as` configuration in the Databricks API. Add a post-deployment permission verification step that confirms the deployed job's service principal has the required Unity Catalog and storage permissions for production data.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 4

Secrets Management

Seven scenarios covering the credential patterns that create single points of failure, silent expiry, and security vulnerabilities, including the Azure Key Vault rotation surface.

23

SECRETS MANAGEMENT

You stored a secret in a Databricks secret scope. It is visible in notebook output logs.

THE SITUATION

A database password is stored in a Databricks secret scope and retrieved in a notebook using

`dbutils.secrets.get()`. The secret value is accidentally printed to a notebook output cell when debugging a connection issue. The printed value is visible in the notebook output and is captured in the notebook execution log.

WRONG THINKING

Engineers store secrets in a secret scope and retrieve them in code, assuming the secret is protected from accidental exposure. The secret scope is expected to prevent the value from being displayed.

WHAT IS ACTUALLY HAPPENING

Databricks secret scopes mask secret values in notebook output by replacing them with REDACTED when the secret is accessed through `dbutils.secrets.get()` and printed directly. However if the secret value is assigned to a variable and then used in string operations such as building a connection string, the constructed string may contain the secret value and print without masking. Connection strings containing embedded credentials are not automatically masked in notebook output.

THE FIX

Never construct connection strings by embedding secret values directly into strings. Use separate username and password parameters in connection APIs rather than embedding secrets in URLs. Avoid any code path that could result in a secret value being included in a string that is printed or logged. After any debugging session where connection failures occurred, review notebook outputs for any potentially exposed credential values and rotate those credentials immediately.

24

SECRETS MANAGEMENT

Your secret scope is backed by Azure Key Vault. The secret rotated and every job broke.

THE SITUATION

An Azure Key Vault-backed secret scope is used in Databricks. Azure Key Vault automatic secret rotation is enabled. When the secret rotates, every Databricks job that reads from the secret scope fails with authentication errors. The rotation happened without any notification to the Databricks platform team.

WRONG THINKING

Engineers use Azure Key Vault for secret management and enable automatic rotation as a security best practice. The Databricks secret scope is expected to automatically pick up the rotated value from Key Vault.

WHAT IS ACTUALLY HAPPENING

Azure Key Vault-backed secret scopes in Databricks read the current secret value from Key Vault at the time the secret is accessed. Automatic rotation changes the Key Vault secret value but any downstream systems using the old value must update. If the external system such as a database has not yet accepted the new credential, jobs fail immediately when they attempt to authenticate with the new value. Additionally if the rotation process involves a period where neither the old nor new value is valid, all jobs fail during that window.

THE FIX

Coordinate secret rotation with the Databricks platform team. Implement a rotation procedure that first validates the new credential against the target system, updates the Key Vault secret, then verifies Databricks jobs can authenticate using the new value before the old credential is invalidated. Configure rotation notifications from Key Vault to alert the Databricks platform team. Test the rotation procedure in a non-production environment before applying to production credentials.

25

SECRETS MANAGEMENT

You granted secret scope access to a group. One user in the group can see all secrets.

THE SITUATION

A Databricks secret scope is shared with a group to allow multiple engineers to access the credentials stored there. An engineer in the group discovers they can list all secrets in the scope and retrieve all values, not just the specific secrets relevant to their work.

WRONG THINKING

Engineers grant group access to a secret scope to avoid managing individual access grants for each engineer. Group access provides the necessary credentials to all team members efficiently.

WHAT IS ACTUALLY HAPPENING

Databricks secret scope access is granted at the scope level rather than the individual secret level. When a group is granted access to a secret scope, all members of that group can access all secrets in that scope. There is no secret-level granularity within a scope. If the scope contains multiple credentials for different systems, all users with scope access can retrieve all of them regardless of which systems they need access to.

THE FIX

Use separate secret scopes for different sensitivity levels or different systems rather than storing all secrets in a single scope. Grant access to individual scopes based on which systems each group or user needs to access. For highly sensitive credentials, use individual secret scopes with explicit per-user grants rather than group grants. Accept that scope-level granularity is a limitation of Databricks secret scopes and design your secret organisation around this constraint.

26

SECRETS MANAGEMENT

You hardcoded a connection string. The password changed and everything broke.

THE SITUATION

A pipeline notebook contains a hardcoded JDBC connection string including the database password. The database team rotates passwords as part of a quarterly security policy. When the password changes, every pipeline using the hardcoded connection string fails simultaneously.

WRONG THINKING

Engineers hardcode connection strings during initial development for simplicity. Fixing the hardcoding is treated as a future task that keeps being deprioritised because the pipelines are working.

WHAT IS ACTUALLY HAPPENING

Hardcoded credentials fail the moment the credential changes. There is no graceful way to update a hardcoded credential without modifying and redeploying every pipeline that contains it. In a workspace with many pipelines, identifying all hardcoded instances of a credential and updating them in a coordinated way is error-prone and time-consuming.

THE FIX

Replace all hardcoded credentials with secret scope references before any pipeline goes to production. Run an automated scan of the workspace for patterns matching common credential formats to identify hardcoded secrets. Treat hardcoded credentials as a blocking code review issue. After migrating to secret scopes, password rotation becomes a single secret scope update rather than a multi-pipeline deployment.

27

SECRETS MANAGEMENT

You moved secrets from notebooks to secret scopes. Developers recreated hardcoded versions because the scope was hard to use.

THE SITUATION

Credentials are migrated from hardcoded notebook values to Databricks secret scopes as a security improvement. Within weeks, developers working on new pipelines are hardcoding credentials again because accessing the secret scope requires additional setup steps that are not documented in the team's onboarding guide.

WRONG THINKING

Platform engineers migrate secrets to the secure store and assume developers will adopt the new pattern. The friction of the new pattern is not considered. Making the secure approach harder to use than the insecure approach ensures the insecure approach will be chosen under time pressure.

WHAT IS ACTUALLY HAPPENING

Security improvements that add friction without reducing it elsewhere are not adopted sustainably. If retrieving a secret from the scope requires more steps than hardcoding it, developers under time pressure will choose the simpler option. The secure infrastructure exists but is not used because it is not the path of least resistance.

THE FIX

Document the secret scope access pattern in a reusable notebook template that all new pipelines start from. Make the correct pattern the default by including it in the project template rather than as optional guidance. Add a linting or code review check that flags hardcoded credential patterns in pull requests. Reduce friction by providing a convenience function that wraps the secret scope retrieval with error handling, making the secure pattern as simple as a single function call.

28

SECRETS MANAGEMENT

Your secret is correct. The job is reading a cached version from 3 days ago.

THE SITUATION

A database password is updated in a Databricks secret scope. Jobs that should use the new password continue failing with authentication errors using the old password. The secret scope shows the correct new value when checked manually.

WRONG THINKING

Engineers update the secret scope and assume all jobs immediately start using the new value. Secret scope updates are expected to take effect immediately for all subsequent secret retrievals.

WHAT IS ACTUALLY HAPPENING

Databricks secret scope values are resolved at job execution time. However if a job cluster is already running and has the old credential cached in a Spark configuration or environment variable set at cluster startup, the running cluster will continue using the old credential until it is restarted. Some credential caching at the application level in JDBC drivers or connection pools may also retain the old credential for the duration of the connection pool lifetime.

THE FIX

Restart all job clusters and running all-purpose clusters after updating a secret scope value to ensure all compute is using the new credential. For connection pool-based applications, implement a credential refresh mechanism that reloads credentials from the secret scope periodically rather than caching them for the process lifetime. After a credential update, monitor the first few job runs to confirm they are authenticating successfully with the new value.

29

SECRETS MANAGEMENT

You deleted a secret scope. Three pipelines that referenced it broke with no warning.

THE SITUATION

A secret scope containing credentials for a decommissioned system is deleted during a cleanup exercise. Three production pipelines that reference secrets from the same scope but connect to different systems begin failing with secret scope not found errors.

WRONG THINKING

Engineers delete the secret scope for the decommissioned system and assume only pipelines for that specific system are affected. The scope is assumed to contain only the relevant credentials.

WHAT IS ACTUALLY HAPPENING

Secret scopes are organisational containers for secrets. If multiple teams added credentials to the same scope over time without a clear naming convention, the scope may contain credentials for many unrelated systems. Deleting the scope removes all contained secrets regardless of which systems they serve. Without a registry of which pipelines use which secret scopes, impact analysis before deletion is impossible.

THE FIX

Before deleting any secret scope, audit which secrets it contains and identify all pipelines, notebooks, and jobs that reference any of those secrets. Use workspace search to find all references to the scope name in code. Migrate any secrets still in active use to a different scope before deleting the original. Implement a secret scope registry that maps each scope to its consumers and maintainers to support future cleanup operations.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 5

Environment Strategy

Seven scenarios covering the gaps between development and production environments that cause failures that were never seen in testing and the configuration drift that accumulates over time.

30

ENVIRONMENT STRATEGY

Your dev environment uses smaller clusters. Code that works in dev breaks in prod due to shuffle behavior.

THE SITUATION

A pipeline runs correctly in development on a 4-node cluster. When promoted to production on a 20-node cluster, the pipeline fails with an out of memory error during a shuffle stage that worked correctly in development.

WRONG THINKING

Engineers test pipelines in development on a representative but smaller cluster and assume the code will behave identically at larger scale. The cluster size difference is considered a performance parameter rather than a correctness parameter.

WHAT IS ACTUALLY HAPPENING

Spark shuffle behavior can differ between cluster sizes in ways that affect correctness. The number of shuffle partitions, memory allocation per executor, and the optimizer's choice of join strategy all depend on cluster size. On a 4-node cluster, data that fits in executor memory without spilling may spill on a 20-node cluster where each executor has less data but different memory pressure due to configuration scaling. Broadcast join thresholds, partition counts, and memory fractions that work on a small cluster may produce different results on a larger one.

THE FIX

Test pipelines in a staging environment that matches production cluster size before promoting to production. Accept that small-scale dev testing cannot guarantee production behavior for all Spark operations. Use parameterized configurations for shuffle partitions and memory settings that match production values even in development. For critical pipelines, run a production-scale validation run in a staging environment before the first production deployment.

31

ENVIRONMENT STRATEGY

You promoted a notebook from dev to prod. It is still reading from dev tables.

THE SITUATION

A notebook is promoted from development to production. The production notebook runs without errors but produces no useful output because it is reading from development tables that contain test data rather than production tables.

WRONG THINKING

Engineers copy the notebook to the production workspace and assume it will automatically read from production data sources because it is in the production workspace. The workspace location is treated as the environment identifier.

WHAT IS ACTUALLY HAPPENING

Notebooks that reference tables using hardcoded two-level or three-level namespace paths will always access those specific tables regardless of which workspace or folder they are deployed to. Moving a notebook to the production workspace does not change the data sources it references. Only explicit path updates in the notebook code or parameterized path references that resolve to different values in different environments will change the data source.

THE FIX

Use widget-based or environment-variable-based table path references in all notebooks that are promoted between environments. Define environment-specific table catalogs and schemas as parameters passed at runtime rather than hardcoded in the notebook. Implement a post-promotion validation that verifies the notebook is accessing production data sources by comparing the referenced catalog and schema against the expected production values.

32

ENVIRONMENT STRATEGY

Your staging environment diverged from production 3 months ago. Nobody noticed.

THE SITUATION

Staging and production environments were in sync when staging was created. After 3 months of production fixes applied directly to production without going through staging, staging is significantly behind production in configuration and code. Staging is no longer a reliable testing environment for production changes.

WRONG THINKING

Engineers apply urgent production fixes directly to production to avoid delay and plan to back-port the fixes to staging later. The back-porting keeps getting deprioritised. Staging gradually drifts.

WHAT IS ACTUALLY HAPPENING

Direct production changes that bypass staging corrupt the purpose of the staging environment. Staging exists to test changes before they reach production. When staging does not reflect production, testing in staging provides false confidence. Changes that pass staging testing may still fail in production because staging's configuration differences mask the real behavior.

THE FIX

Establish a policy that all changes go through staging before production regardless of urgency. For truly urgent changes, apply them simultaneously to both staging and production rather than skipping staging. After any direct production fix, immediately update staging to match. Perform a quarterly environment sync review that compares staging and production configurations and resolves all differences. Treat staging divergence as a technical debt item with the same priority as other debt.

33

ENVIRONMENT STRATEGY

You have no staging environment. Every prod deployment is a live experiment.

THE SITUATION

There is no staging environment. All code changes are deployed directly to production after testing in development. On average, one production deployment per week causes a pipeline failure or data quality issue that requires investigation and remediation.

WRONG THINKING

Engineers skip staging to reduce infrastructure cost and management overhead. Development testing is assumed to be sufficient validation before production deployment. The cost of production incidents from untested deployments is not calculated against the cost of maintaining a staging environment.

WHAT IS ACTUALLY HAPPENING

Direct production deployment from development treats every deployment as a live experiment on production data and business users. Issues that only appear at production scale, with production data volumes, or with production service principal permissions are not caught before they affect business users. The cumulative cost of one incident per week in engineer time and business disruption significantly exceeds the cost of maintaining a staging environment.

THE FIX

Create a minimal staging environment that mirrors production in configuration but not necessarily in data volume. A staging environment does not require full production data to catch most deployment issues. Use a representative sample of production data in staging to catch scale-related issues. The minimum viable staging environment requires only a workspace, a set of representative job definitions, and service principals configured equivalently to production.

34

ENVIRONMENT STRATEGY

Your environment promotion is manual. The last 5 promotions each had at least one error.

THE SITUATION

Code changes are promoted from development to production through a manual process that involves copying notebooks, recreating job configurations, and updating table references. The last 5 production deployments each had at least one error requiring a hotfix within 24 hours of promotion.

WRONG THINKING

Engineers accept manual promotion errors as the cost of the current process. Each error is investigated individually without recognising the pattern of systematic manual error.

WHAT IS ACTUALLY HAPPENING

A manual promotion process that consistently produces errors is a process problem, not a people problem. If every promotion introduces errors, the process is not reliable and the team is spending more time on hotfixes than would be spent on automation. Manual processes are limited by human attention and memory. Automated processes enforce completeness and consistency on every execution.

THE FIX

Invest in automating the promotion process using Asset Bundles or equivalent tooling. The time invested in automation pays back through eliminated hotfixes within a few months for a team with weekly deployments. Document every step of the current manual process explicitly as the specification for the automated process. The documentation exercise itself often surfaces steps that were being inconsistently applied in the manual process.

35

ENVIRONMENT STRATEGY

Dev and prod use different Databricks Runtime versions. A bug only appears in production.

THE SITUATION

Development clusters run DBR 14. Production clusters run DBR 13 for stability. A data engineer introduces code that uses a function available in DBR 14 but not in DBR 13. Development testing passes. The first production run fails immediately with a function not found error.

WRONG THINKING

Engineers run the latest runtime in development for access to new features and keep production on a stable older version. The function availability difference between the two runtimes is not checked before using a new function in development.

WHAT IS ACTUALLY HAPPENING

Runtime version differences between development and production mean that code tested against one runtime may fail against another. Function availability, default configuration values, type handling, and optimizer behavior can all differ between runtime versions. Development code that uses features specific to a newer runtime will fail in production without any indication during development testing.

THE FIX

Align development and production runtime versions to eliminate version-specific behavioral differences. When a runtime upgrade is planned for production, upgrade development first and test thoroughly before upgrading production. For teams that want access to new features in development before production is ready to upgrade, document the list of features that are not yet available in production and add a code review check that flags usage of those features in code intended for production deployment.

36

ENVIRONMENT STRATEGY

Your prod environment has config that was changed manually 6 months ago. Nobody knows why.

THE SITUATION

A production cluster has a non-standard Spark configuration that differs from the standard template. Investigation reveals the configuration was changed manually 6 months ago with no documented reason. Nobody currently on the team knows why the change was made or whether removing it would break anything.

WRONG THINKING

Engineers make production configuration changes when debugging incidents and do not document the reason or expected lifetime of the change. Emergency fixes are applied and then forgotten.

WHAT IS ACTUALLY HAPPENING

Undocumented production configuration changes create technical debt that compounds over time. The original context for the change is lost, making it impossible to evaluate whether the change is still needed. Every subsequent incident investigation is complicated by the presence of non-standard configurations whose purpose is unknown. Over time, production configuration becomes a collection of historical fixes that nobody understands.

THE FIX

Require all production configuration changes to be made through code-reviewed configuration files or deployment pipelines rather than manual UI changes. For emergency manual changes, require documentation of the reason and expected resolution date within 24 hours. Conduct quarterly configuration audits that compare production configurations against the documented standard template and remove undocumented deviations after investigation.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 6

Monitoring and Observability

Seven scenarios covering the specific ways Databricks pipelines fail silently while appearing healthy and the monitoring gaps that allow data quality and reliability issues to reach business users before engineers.

37

MONITORING AND OBSERVABILITY

Your Databricks job failed. You found out from a business user 4 hours later.

THE SITUATION

A critical morning ETL job fails at 7am. No alerts fire. The data engineering team is not aware of the failure. A business user calls at 11am to report that the daily dashboard is showing yesterday's data. The 4-hour gap between failure and detection caused downstream business impact.

WRONG THINKING

Engineers configure Databricks job failure notifications and assume they will be alerted promptly when jobs fail. The notification system is trusted to deliver timely alerts.

WHAT IS ACTUALLY HAPPENING

As covered in the Workflows book, Databricks email notifications are not guaranteed to deliver within a defined timeframe. The 4-hour gap indicates either notification delivery delay or notification misconfiguration. For morning SLA-bound jobs, email-based alerting alone is insufficient for timely incident detection.

THE FIX

Implement a proactive monitoring check that runs on a schedule and queries the Databricks Jobs API to verify that SLA-bound jobs have completed successfully within their expected window. If a job has not completed by its expected completion time, the monitoring check fires an alert through a reliable channel such as Slack or PagerDuty rather than email. Treat SLA-bound job monitoring as a separate concern from job failure notification.

38

MONITORING AND OBSERVABILITY

Your job succeeded but processed half the expected rows. No alert fired.

THE SITUATION

A daily pipeline runs and completes with Success status. A business user notices that yesterday's totals are 50 percent lower than expected. Investigation reveals the pipeline processed only half the expected source rows due to an upstream data delivery delay. The job succeeded in terms of execution but delivered incomplete data.

WRONG THINKING

Engineers configure job success and failure monitoring and assume a successful job delivered complete data. Output volume monitoring is not implemented because the job has always processed the full dataset in the past.

WHAT IS ACTUALLY HAPPENING

Job execution success is a binary signal: the job either raised an exception or it did not. It says nothing about whether the output volume meets business expectations. A pipeline that successfully processes half the expected data is a data quality failure that Databricks monitoring does not detect unless explicit data volume checks are implemented.

THE FIX

Add output volume assertions to all critical pipelines as mandatory pipeline steps. After writing to each target table, check the row count against an expected minimum threshold and fail the pipeline explicitly if the count is below threshold. Use historical row counts to define dynamic thresholds rather than static values to accommodate natural data volume variation. Configure threshold breach alerts through the same channel as job failure alerts.

39

MONITORING AND OBSERVABILITY

You set up Databricks system tables for monitoring. The query costs more than the jobs it monitors.

THE SITUATION

Databricks system tables are configured as the monitoring data source for a pipeline health dashboard. The monitoring queries run frequently to keep the dashboard current. After one month, the monitoring queries against system tables are consuming a significant fraction of the SQL warehouse capacity and adding measurable cost.

WRONG THINKING

Engineers use system tables as a convenient native monitoring data source without profiling the cost of frequent queries against them. System tables are assumed to be optimised for the monitoring use case.

WHAT IS ACTUALLY HAPPENING

Databricks system tables store operational metadata in Delta tables that can be queried like any other Delta table. Frequent queries against large system tables such as the job run history or cluster events can be expensive if the tables are large and the queries are not well-optimised. For monitoring dashboards that refresh frequently, each refresh submits a potentially expensive system table query.

THE FIX

Optimise system table monitoring queries by filtering to recent time windows and specific job IDs rather than full table scans. Cache aggregated monitoring metrics in a separate summary table that is refreshed on a defined schedule rather than querying system tables on every dashboard refresh. For high-frequency monitoring requirements, consider using the Databricks REST API to query job status rather than system tables, as API queries do not consume SQL warehouse capacity.

40

MONITORING AND OBSERVABILITY

Your cluster logs are stored. Finding the relevant log for a specific failure takes 30 minutes.

THE SITUATION

Cluster logs are configured to be shipped to an Azure Storage account. When a failure occurs, the on-call engineer spends 30 minutes finding the relevant log file among thousands of files organised by cluster ID and timestamp before they can begin investigating the actual error.

WRONG THINKING

Engineers implement log storage as the log management solution. Storing logs is treated as equivalent to having observability. The discoverability of those logs during an incident is not considered.

WHAT IS ACTUALLY HAPPENING

Log storage without log search and correlation infrastructure provides durability but not observability. In an incident, the most valuable capability is finding the relevant log quickly. A flat directory structure organised by cluster ID requires knowing which cluster to look at before you can find the relevant log. Without a log search layer, incident response time is dominated by log navigation rather than root cause analysis.

THE FIX

Implement a log search solution that indexes cluster logs and allows queries by job ID, error message, timestamp range, and cluster ID. On Azure, configure the workspace's Diagnostic Settings (under Monitoring in the Azure portal) to deliver cluster events, job runs, audit logs, and driver/worker logs to a Log Analytics workspace, where they become queryable via KQL with sub-second response times. Logs can be routed in parallel to an Azure Storage account for long-term archival and to an Event Hub for streaming into a SIEM. Investing in Log Analytics indexing reduces incident mean time to resolution significantly. For each logged event, ensure the job ID and run ID are included in the log context so that any log entry can be traced back to the specific job run that generated it. Note that diagnostic settings on Azure Databricks require the Premium plan.

41

MONITORING AND OBSERVABILITY

Job duration alerts are configured. They fire after the job finishes not when it starts running long.

THE SITUATION

Job duration alerts are configured to fire when a job exceeds its expected duration. The alert fires after the job completes, at which point the SLA has already been missed. The alert provides notification that a miss occurred but does not provide the opportunity to intervene before the miss.

WRONG THINKING

Engineers configure job duration alerts expecting them to fire while the job is still running, providing time to investigate and potentially abort the job before the SLA deadline.

WHAT IS ACTUALLY HAPPENING

Databricks job duration alerts typically evaluate at job completion or at periodic intervals. If the alert checks at completion and the completion time exceeds the threshold, the alert fires after the fact. To alert during execution, an external monitoring system must continuously poll the job run duration using the Databricks Jobs API and compare against the expected duration threshold while the job is still running.

THE FIX

Implement a separate proactive monitoring job that runs every few minutes and queries active job run durations via the Databricks REST API. If any active job exceeds its expected duration by a defined threshold, the monitoring job fires an immediate alert. This in-flight monitoring enables intervention before SLA misses rather than notification after. Configure expected durations per job based on historical run time distributions.

42

MONITORING AND OBSERVABILITY

You have 200 jobs. There is no single view showing which ones are healthy and which are not.

THE SITUATION

The Databricks platform has grown to 200 jobs across multiple workspaces. There is no consolidated view of job health. On-call engineers check job health by navigating through multiple workspace job history views. During an incident, assessing overall platform health requires manually checking multiple workspaces and cannot be done quickly.

WRONG THINKING

Engineers manage jobs in the workspace UI which shows job history per workspace. As the number of jobs and workspaces grows, the workspace UI remains the primary monitoring interface. A consolidated cross-workspace view is considered a future improvement.

WHAT IS ACTUALLY HAPPENING

The Databricks workspace UI shows job history per workspace but does not provide a cross-workspace operational dashboard. For organisations with multiple workspaces and many jobs, no native consolidated health view exists. Manual health checks scale linearly with the number of workspaces and jobs, making them impractical for incident response.

THE FIX

Build a consolidated monitoring dashboard using the Databricks system tables and Jobs API that shows the health status of all jobs across all workspaces in a single view. Categorise jobs by SLA tier and show the last run status, duration, and any threshold violations for each job. This dashboard becomes the first point of reference during incidents, reducing the time required to assess platform health from minutes to seconds.

43

MONITORING AND OBSERVABILITY

Your monitoring dashboard shows all jobs green. Three jobs are processing zero rows.

THE SITUATION

A monitoring dashboard shows all 50 production jobs with green status indicators. A business user reports that three specific reports have shown no new data for two days. Investigation reveals the three corresponding jobs are completing successfully but writing zero rows to their target tables. The monitoring dashboard shows green because it monitors job execution status, not data output.

WRONG THINKING

Engineers build a monitoring dashboard around job execution status as the primary health signal. A green job status means a healthy pipeline. Data output validation is assumed to be covered by job success status.

WHAT IS ACTUALLY HAPPENING

As established throughout both series, job execution status and data output correctness are independent signals. A green status means execution completed without exception. It says nothing about data volume, data quality, or whether the expected rows were processed. A monitoring system that only tracks execution status will always show green for zero-row failures.

THE FIX

Extend the monitoring dashboard to include data freshness metrics alongside execution status. For each monitored table, show the last update timestamp and the row count from the last pipeline run. Configure alerts when the last update timestamp exceeds the expected update frequency. A dashboard that shows both execution status and data freshness provides the full picture needed to confirm pipeline health from both an operational and business perspective.

AZURE DATABRICKS ENGINEERING PATTERNS / BOOK 7

Chapter 7

Access Control and Identity

Seven scenarios covering the service principal and user credential patterns that create security vulnerabilities, single points of failure, and audit gaps, including the Microsoft Entra ID and IP access list paths.

44

ACCESS CONTROL AND IDENTITY

A developer left the company. Their personal PAT token is still used in 12 production jobs.

THE SITUATION

A developer who left the company 3 months ago had their Databricks account deactivated. Investigation during an incident reveals that 12 production jobs are still configured to use that developer's personal access token for authentication. These jobs have been failing for 3 months due to the deactivated token.

WRONG THINKING

Engineers configure jobs to run using personal tokens during development and promote them to production without migrating to service principals. The token is embedded in the job configuration and is not reviewed during offboarding.

WHAT IS ACTUALLY HAPPENING

Personal access tokens are tied to the individual user's account. When an account is deactivated, all tokens associated with that account are immediately invalidated. Any automated job using a personal token fails upon account deactivation. Personal tokens in production jobs create a brittle dependency between the job's availability and the individual's continued employment and active account status.

THE FIX

Require all production jobs to run as service principals rather than personal user identities. Service principal tokens do not expire when individuals leave the company. Implement an offboarding checklist that includes auditing all workspace jobs for personal token usage and migrating them to service principals before the individual's account is deactivated. Run a quarterly audit that identifies any jobs using personal tokens and migrates them proactively.

45

ACCESS CONTROL AND IDENTITY

Your service principal has workspace admin rights. It was provisioned that way because it was easier.

THE SITUATION

A service principal used for automated pipeline execution has workspace admin permissions. The admin permission was granted during initial setup to avoid having to figure out the minimum required permissions. Months later, the service principal's credentials are leaked in a code repository. The attacker has workspace admin access to the entire Databricks platform.

WRONG THINKING

Engineers grant workspace admin to service principals for convenience during setup and plan to reduce permissions later. Reducing permissions is deprioritised because the current setup is working. The security risk of over-provisioned service principals is accepted implicitly.

WHAT IS ACTUALLY HAPPENING

Over-provisioned service principals create disproportionate risk when their credentials are compromised. A service principal with workspace admin access can create new users, modify access controls, access all data in the workspace, and extract credentials. A least-privilege service principal limited to the specific jobs it needs to run has a much smaller blast radius when compromised.

THE FIX

Provision all service principals with the minimum permissions required for their specific workloads. Define a permission profile for each service principal based on what it needs to read, write, and execute. Grant only those specific permissions rather than workspace admin. Rotate service principal credentials immediately when any code repository containing credentials is exposed. Implement a regular permission audit that flags service principals with admin or over-broad permissions for review and reduction.

46

ACCESS CONTROL AND IDENTITY

You granted a user CAN MANAGE on a job. They deleted it by accident.

THE SITUATION

A data engineer is granted CAN MANAGE permission on a production job to allow them to update the job schedule. Using the same permissions, the engineer accidentally deletes the job while exploring the job management options. The job configuration is lost and must be reconstructed from documentation.

WRONG THINKING

Engineers grant CAN MANAGE permission to allow job schedule updates, assuming CAN MANAGE is limited to management operations like schedule changes. Delete is assumed to require a more specific permission.

WHAT IS ACTUALLY HAPPENING

CAN MANAGE in Databricks job permissions includes the ability to delete the job. There is no intermediate permission level that allows schedule and configuration updates without also allowing deletion. Engineers who need to manage job schedules but should not be able to delete jobs cannot be granted the minimum permission they need without also granting deletion capability.

THE FIX

For production jobs, grant CAN MANAGE only to a small set of senior engineers and platform admins. For engineers who need to update job schedules, use Databricks Asset Bundles to manage job configurations as code. Schedule updates are then made by updating the bundle configuration and running a deployment rather than through direct workspace UI access. This removes the need for CAN MANAGE on production jobs for most engineers while providing a safer configuration management path.

47

ACCESS CONTROL AND IDENTITY

Your SCIM sync is configured. User deprovisioning takes 24 hours to take effect.

THE SITUATION

SCIM provisioning is configured to sync user accounts from the identity provider to Databricks. When a user is removed from the identity provider, their Databricks account is deprovisioned. However the deprovisioning takes up to 24 hours to complete due to the SCIM sync frequency. During this window the user retains full Databricks access.

WRONG THINKING

Engineers implement SCIM and assume deprovisioning is immediate when a user is removed from the identity provider. SCIM sync is treated as providing real-time access control.

WHAT IS ACTUALLY HAPPENING

SCIM sync operates on a polling interval configured in the identity provider's provisioning settings. If the interval is set to 24 hours or 12 hours, user account changes including deprovisioning are not reflected in Databricks until the next sync cycle. For terminated employees who should lose access immediately upon departure, a 24-hour sync window is a significant security gap.

THE FIX

Recognise that Microsoft Entra ID's SCIM provisioning interval is fixed at 20 to 40 minutes by Microsoft and cannot be configured shorter, so reducing the interval is not a real option. For immediate access revocation such as termination for cause, do not wait for SCIM; deactivate the Databricks account directly via the account console or the Databricks SCIM API, in addition to disabling the user in Entra ID. Consider enabling Databricks automatic identity management, which can shorten the perceived propagation window for some operations (browser logins refresh group memberships if more than five minutes have passed since the last sync; other activities like token authentication or running jobs sync if more than 40 minutes have passed). Document the manual deprovisioning process in the security incident response playbook and test it regularly. Monitor the time between identity provider deprovisioning and Databricks account deactivation as a security metric.

48

ACCESS CONTROL AND IDENTITY

You use Microsoft Entra groups for access control. A group change took 6 hours to propagate.

THE SITUATION

Access to a Unity Catalog catalog is granted to a Microsoft Entra group. A new engineer is added to the Entra group. The engineer cannot access the catalog for 6 hours after being added to the group. The group membership change is not reflected in Databricks Unity Catalog until the next group sync cycle.

WRONG THINKING

Engineers grant Unity Catalog access to Entra groups expecting group membership changes to propagate immediately. Adding a user to a group should immediately grant that user all permissions the group holds.

WHAT IS ACTUALLY HAPPENING

Unity Catalog group membership is synchronised from the external identity provider through periodic SCIM sync. The frequency of this sync determines how quickly group membership changes propagate to Unity Catalog permission evaluations. A 6-hour propagation delay means new team members cannot access required data for half a working day after being added to the group.

THE FIX

Recognise that the SCIM provisioning interval from Microsoft Entra ID is fixed at 20 to 40 minutes by Microsoft and cannot be reduced through configuration. For new hire onboarding where immediate access is required, implement a temporary direct grant on the user as a Databricks-side workaround that is removed once the group membership propagates through SCIM. Consider enabling Databricks automatic identity management for the account, which refreshes group memberships within five minutes for browser logins (still 40 minutes for token-authenticated activity such as running jobs). Document the expected propagation delay for group changes in team onboarding materials so new engineers and their managers have accurate expectations. Monitor actual sync propagation time and alert when it exceeds 40 minutes, which would indicate sync failures rather than normal lag.

49

ACCESS CONTROL AND IDENTITY

Your service principal tokens never expire. A leaked token has been active for 2 years.

THE SITUATION

A service principal access token generated 2 years ago is discovered in a public code repository commit. The token was never rotated and is still active. Investigation shows the token has full workspace access and has been accessible to anyone who found it in the repository for 2 years.

WRONG THINKING

Engineers create service principal tokens without expiry for operational convenience. Tokens that expire require rotation which creates operational overhead. Long-lived or non-expiring tokens seem more reliable for automation.

WHAT IS ACTUALLY HAPPENING

Non-expiring tokens create permanent exposure windows when leaked. A token that never expires and is discovered in a code repository remains a valid credential indefinitely. The longer a leaked token is active, the longer an attacker has had time to use it and the harder it is to determine the scope of any potential unauthorized access.

THE FIX

Set maximum token lifetimes for service principals using Databricks workspace token policies. Configure tokens to expire after 90 days or less for production service principals. Implement automated token rotation that generates new tokens before expiry and updates dependent job configurations. Scan code repositories for credential patterns using secret scanning tools. Treat any credential found in a repository as compromised and rotate it immediately regardless of whether unauthorized access is confirmed.

50

ACCESS CONTROL AND IDENTITY

You restricted workspace access. A contractor found an old bookmark that still works.

THE SITUATION

Workspace access restrictions are tightened to require corporate network or specific IP addresses. A contractor whose access was intended to be restricted accesses the workspace through an old bookmarked URL that bypasses the new IP restriction because IP access lists were configured at the wrong scope or because the contractor's token authenticated through a path the IP access list does not cover.

WRONG THINKING

Engineers apply IP-based access restrictions to what they believe is the complete workspace access path. The IP access list is configured at the workspace level. Alternative access paths including the Azure Databricks account console, direct REST API calls, and Private Link endpoints are not considered, and the distinction between front-end and back-end network paths is not understood.

WHAT IS ACTUALLY HAPPENING

Azure Databricks workspaces can be accessed through multiple paths including the web UI, REST API, JDBC connections, and various SDK integrations. Workspace-level IP access lists restrict the workspace endpoint but do not cover the Azure Databricks account console (which has its own separate IP access list configuration). Additionally Microsoft Learn explicitly notes that IP access lists apply only to requests over the public internet: traffic arriving through a Private Link private endpoint cannot be blocked by an IP access list because it never carries a public source IP. If the contractor has a Personal Access Token or a service principal token issued before the restriction was applied, that token still authenticates through any allowed path including Private Link.

THE FIX

Layer the controls. Apply workspace-level IP access lists for the public internet path, and configure a separate account-console IP access list to cover account-level operations. For environments that require strict network isolation, disable public network access on the workspace entirely and route all traffic through Azure Private Link front-end private endpoints, then restrict who can reach those endpoints using Azure NSGs and your network security perimeter. Independently of network restrictions, revoke all Personal Access Tokens and service principal tokens for the departing contractor; tokens already issued are not invalidated by IP access list changes. Test the restrictions explicitly by attempting access from an unauthorised path before declaring the configuration complete.

CONCLUSION

The platform you do not see is the one running you.

Across 50 scenarios the pattern repeats. One workspace shared by every team becomes a coordination tax that scales with headcount. Idle clusters cost more than the optimised jobs that run on them. Asset Bundle deployments succeed silently while using the wrong configuration. Secrets are stored in scopes that developers route around because the scope is harder to use than a hardcoded string. Staging environments drift from production until staging stops being a useful test. SCIM sync takes hours to deprovision a departing employee. The lesson is that the Databricks platform is not the workspace UI; it is the topology, the cost model, the deployment pipeline, the secret store, the network perimeter, and the identity layer. Each of these is a separate system with its own failure modes, and a platform is only as reliable, governed, and secure as the weakest of them.

ALSO IN THE SERIES

Clusters and Compute

Your job failed in 30 seconds. The cluster spent 28 of them starting up.

Unity Catalog

You created an external location. Users can still access the storage directly.

Databricks SQL and Photon

Your SQL warehouse is set to X-Large. Queries are no faster than on Medium.

And more. Download the full series free at ssanjaychandra.com

A NOTE ON THIS RELEASE

This is Book 7 in the Azure Databricks Engineering Patterns series, and the final book in the seven-book set. The platform and cost mechanics described here are largely the same on every cloud, with Azure-specific context introduced where the platform genuinely changes the answer: workspace migration across Azure regions, Azure Cost Management as the consumption view for DBU and VM charges, Azure VM tag propagation timing, Log Analytics workspaces as the destination for Diagnostic Settings, and the layered controls of workspace-level IP access lists, account-console IP access lists, and Azure Private Link front-end private endpoints. Databricks and Azure both evolve quickly and some guidance, particularly around the IP access list and Private Link interaction, may need updating as the platforms mature. If you spot a technical inaccuracy or a scenario that deserves deeper treatment, your feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this book together.